

Poker DQN — 9.3M Training Run Analysis

Push/Fold Range Evolution, ICM Module Evaluation,
and Strategic Behavior Assessment

Training run: Post-bugfix run (20260416_155300_916856)

Duration: 9,300,000 tournaments (checkpoints every 100k)

Agents: 4 independent Poker_DQN agents (no weight sharing)

Architecture: ICMNet(8→128→4, softmax) + DuelingDQN(12→128→128→2)

Action space: Binary: all-in or fold

Prize pool: [+1.5, +0.5, -0.5, -1.5] (zero-sum)

Date: 2026-04-20

Table of Contents

1. Executive Summary
2. Push Frequency Progression (0 → 9.3M)
3. Range Card Snapshots (2M / 4.5M / 7M / 9.3M)
4. ICM Module Analysis
 - 4.1 Stack Sweep
 - 4.2 Scatter Plot
 - 4.3 Decision Impact
 - 4.4 Property Comparison
5. Key Findings and Interpretation
6. Recommendations

1. Executive Summary

This report analyzes the complete 9.3M tournament training run of four independent Poker_DQN agents playing push/fold poker. Key findings:

Range evolution: All agents converge from wide, noisy ranges (40-50% push at 100k) to tight, hand-strength-correlated ranges by 9.3M. Agents 1-3 settle at 1-8% push frequency (very tight, premium-only), while Agent 4 remains the loosest at ~15%, exploiting the tight opponents.

Continued learning 6M→9.3M: Agents 2 and 3 show interesting late-training dynamics. Agent 2 loosens from 1% to 11% push between 7M and 9.3M, and Agent 3 rebounds from 2% to 8%. This suggests strategy adaptation is still ongoing and equilibrium has not been reached.

ICM module: The learned ICMNet has collapsed to constant one-hot outputs for all agents, providing no stack-dependent equity information. The DQN makes decisions almost entirely from the 8-dim base state. Agents 2 and 4 show 100% push agreement when swapping learned ICM for exact ICM. Agent 3 is a notable exception: only 3.6% agreement, indicating its DQN has uniquely adapted to use the collapsed ICM value as a meaningful signal.

2. Push Frequency Progression

Push frequency is tracked at every 100k checkpoint, broken down by position (4 positions, position 0 = earliest to act, position 3 = latest). The plot covers the full 0→9.3M training trajectory.

Push Frequency Progression — Stack=25.0BB, Active=4, Shortest=True, Mode=softmax T=1.0

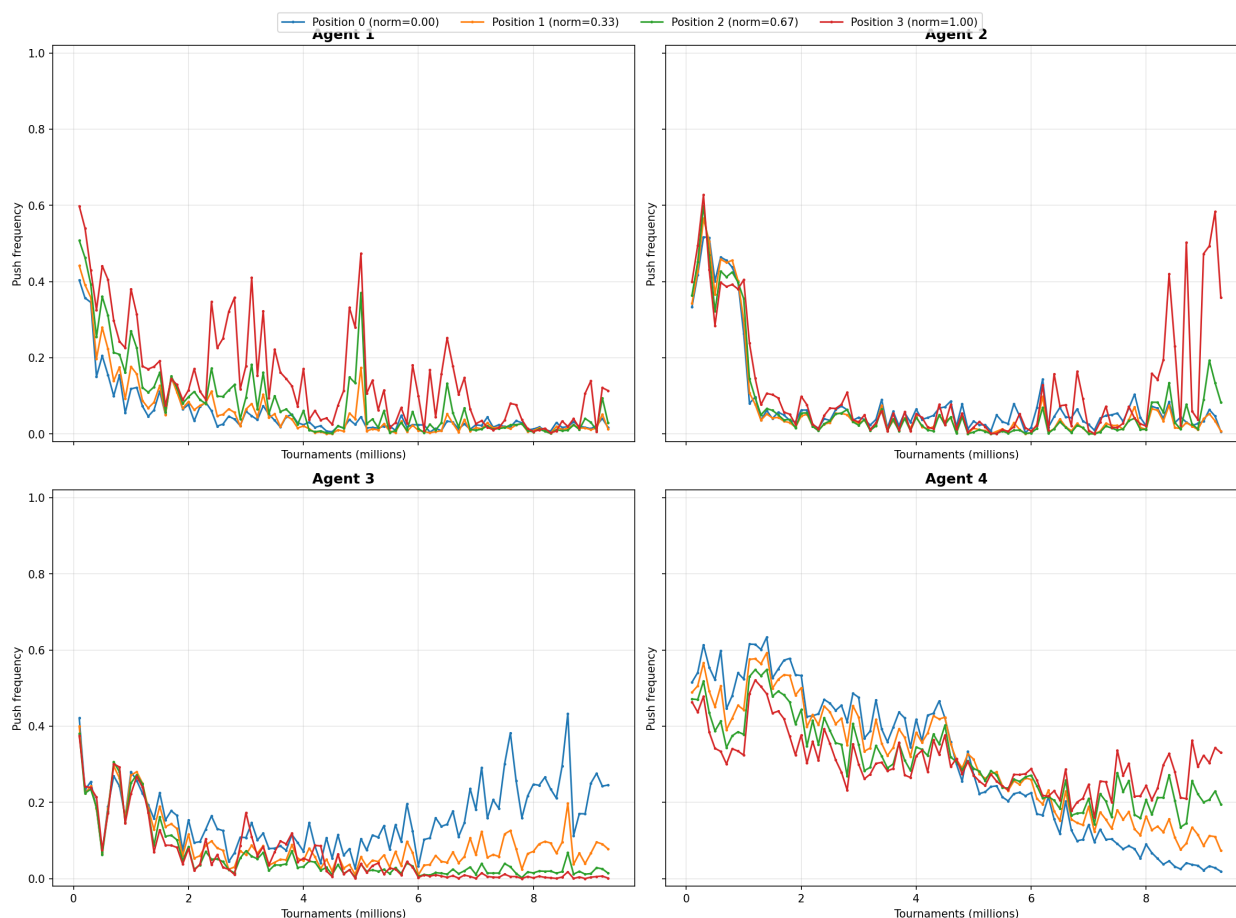


Figure 1: Push frequency vs training tournaments, per position.

Agent 1: Rapidly tightens from ~40% to under 5% by 2M. Remains very tight (1-4%) through 9.3M. Positions are closely clustered, suggesting limited positional awareness.

Agent 2: Similar rapid tightening to 1-4% by 3M. A notable late divergence occurs after 7M where position 3 (latest position) rises sharply, indicating emerging positional adaptation — the agent learns to push wider from late position.

Agent 3: Tightens to 1-3% by 4M, then shows rebound activity after 5M. By 9.3M, push frequency is 5-10% with some position differentiation starting to emerge.

Agent 4: The loosest agent throughout. Starts at ~50%, drops to 10-15% by 5M, then stabilizes. Shows the most positional spread, pushing wider from later positions. This is strategically sound — exploiting the tight opponents.

3. Range Card Snapshots

13×13 heatmaps of $P(\text{all-in})$ for every starting hand, averaged across all 4 positions, at 4 milestones: 2M, 4.5M, 7M, 9.3M. Green = high push probability, red = fold. Parameters: stack=25BB, 4 active players, shortest stack, softmax $T=1.0$.

3.1 Agent 1

Agent 1 — Range Snapshots (position avg)
Stack=25.0BB, Active=4, Shortest=True, Mode=softmax $T=1.0$

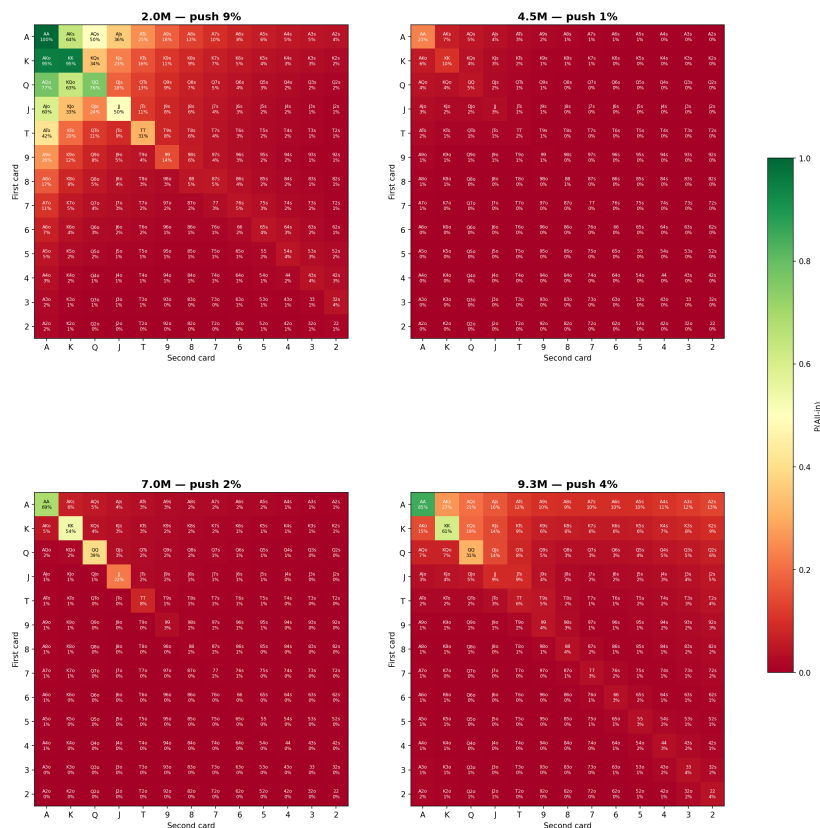


Figure 2: Agent 1 range snapshots at 2M / 4.5M / 7M / 9.3M.

3.2 Agent 2

Agent 2 — Range Snapshots (position avg)
Stack=25.0BB, Active=4, Shortest=True, Mode=softmax T=1.0

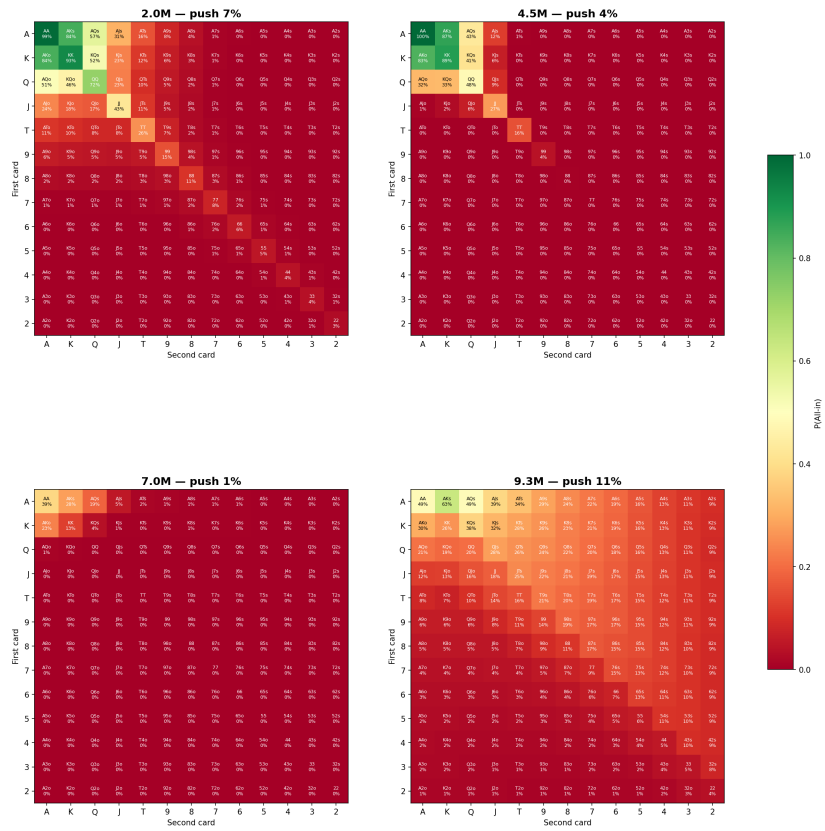


Figure 3: Agent 2 range snapshots at 2M / 4.5M / 7M / 9.3M.

3.3 Agent 3

Agent 3 — Range Snapshots (position avg)
Stack=25.0BB, Active=4, Shortest=True, Mode=softmax T=1.0

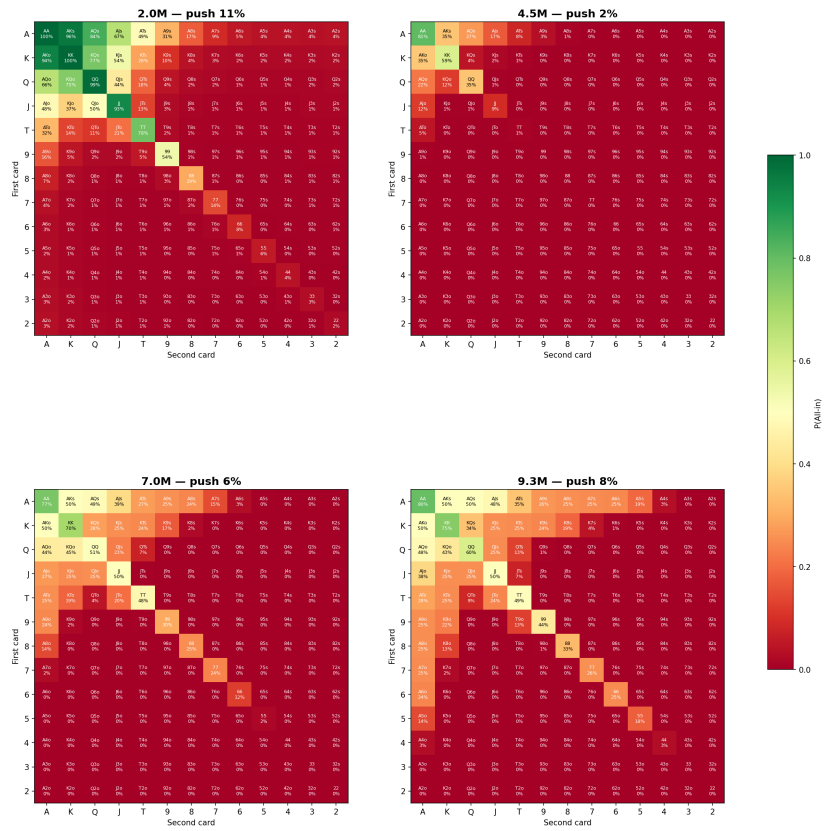


Figure 4: Agent 3 range snapshots at 2M / 4.5M / 7M / 9.3M.

3.4 Agent 4

Agent 4 — Range Snapshots (position avg)
Stack=25.0BB, Active=4, Shortest=True, Mode=softmax T=1.0

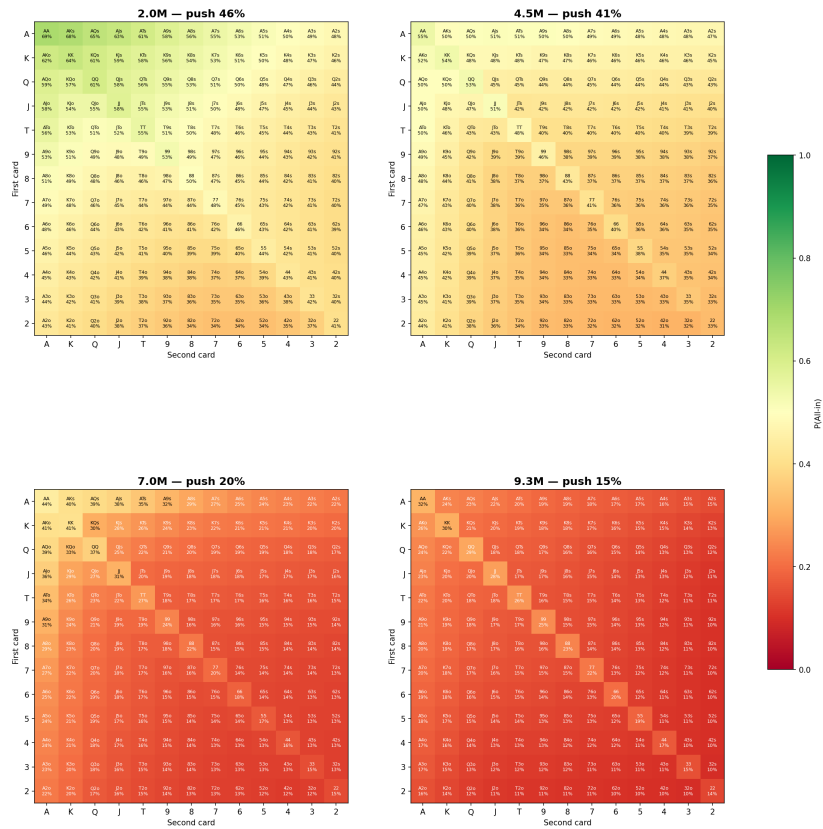


Figure 5: Agent 4 range snapshots at 2M / 4.5M / 7M / 9.3M.

Range Card Observations:

Agent	2M	4.5M	7M	9.3M	Trend
Agent 1	9%	1%	2%	4%	Very tight; only premium pairs + AK/AQ
Agent 2	7%	4%	1%	11%	Tightens then loosens; late adaptation
Agent 3	11%	2%	6%	8%	V-shaped; rebounds with wider suited range
Agent 4	46%	41%	20%	15%	Consistently loosest; clear hand-strength gradient

Agent 4 shows the clearest hand-strength gradient throughout training: high pairs and Broadway hands are pushed most, with probability decreasing smoothly toward low cards. This is the expected pattern for rational play.

Agents 2 and 3 exhibit a "V-shaped" trajectory: they tighten aggressively in mid-training (4.5M-7M) then loosen again by 9.3M. This suggests the agents are still adapting to each other's strategies and equilibrium has not been reached.

4. ICM Module Analysis

The ICMNet (Linear(8,128)→ReLU→Linear(128,4)→Softmax) is intended to learn tournament equity from chip stacks. It is trained **indirectly** via the DQN's TD-error loss — no supervised ICM targets. We compare its outputs against exact Malmuth-Harville ICM at the 9.3M checkpoint.

4.1 Stack Sweep

Player 0's stack is swept from 1BB to 95BB (others fixed at 25BB). Left axis: learned ICMNet output. Right axis: exact ICM equity.

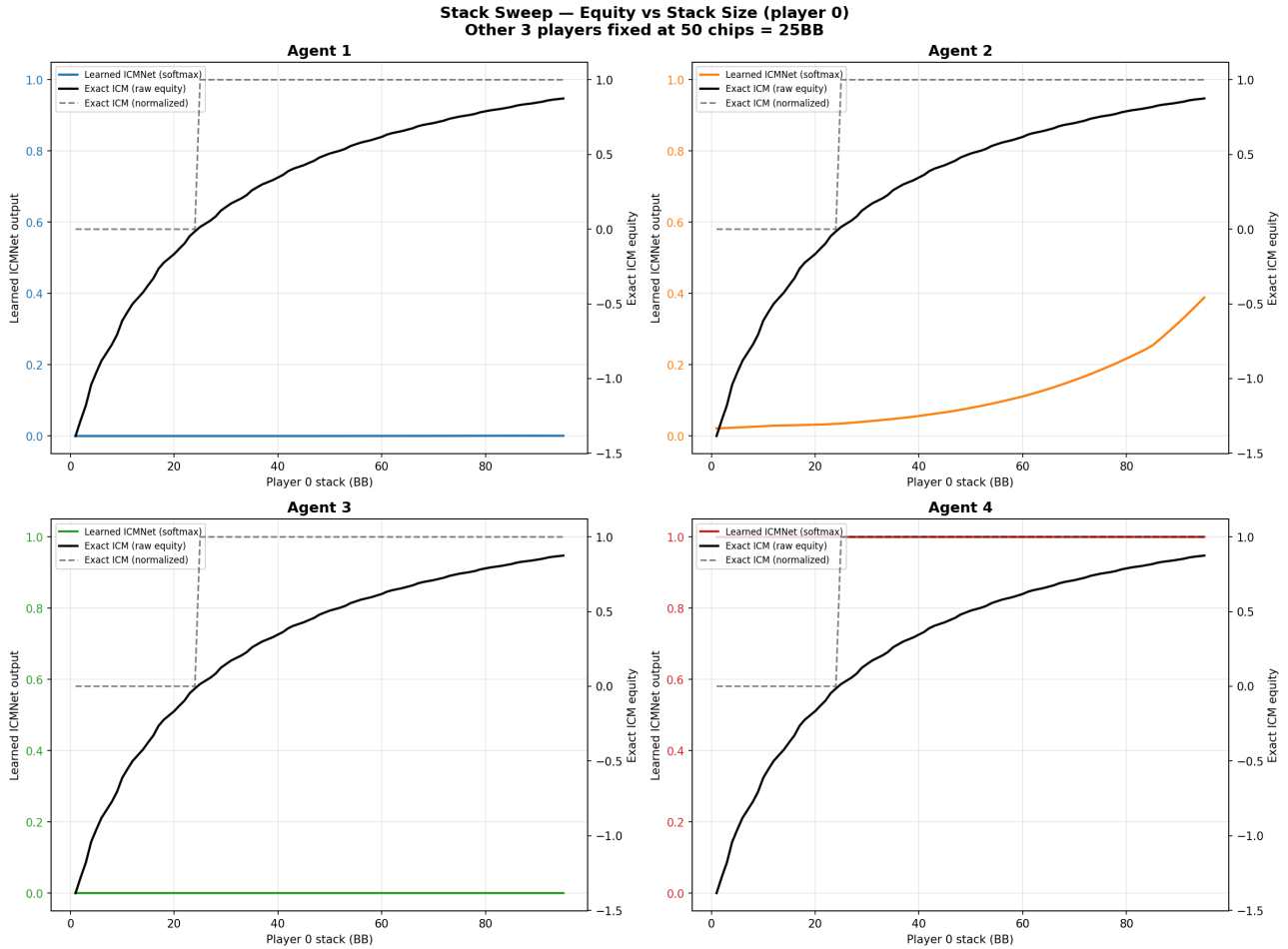


Figure 6: Stack sweep — equity vs stack size at 9.3M.

Exact ICM (black) shows the expected concave monotonic curve. All 4 learned ICMNets output near-constant values regardless of stack size — the same collapse pattern observed at 6.3M persists through 9.3M. Additional training did not recover the ICM module.

4.2 Scatter Plot

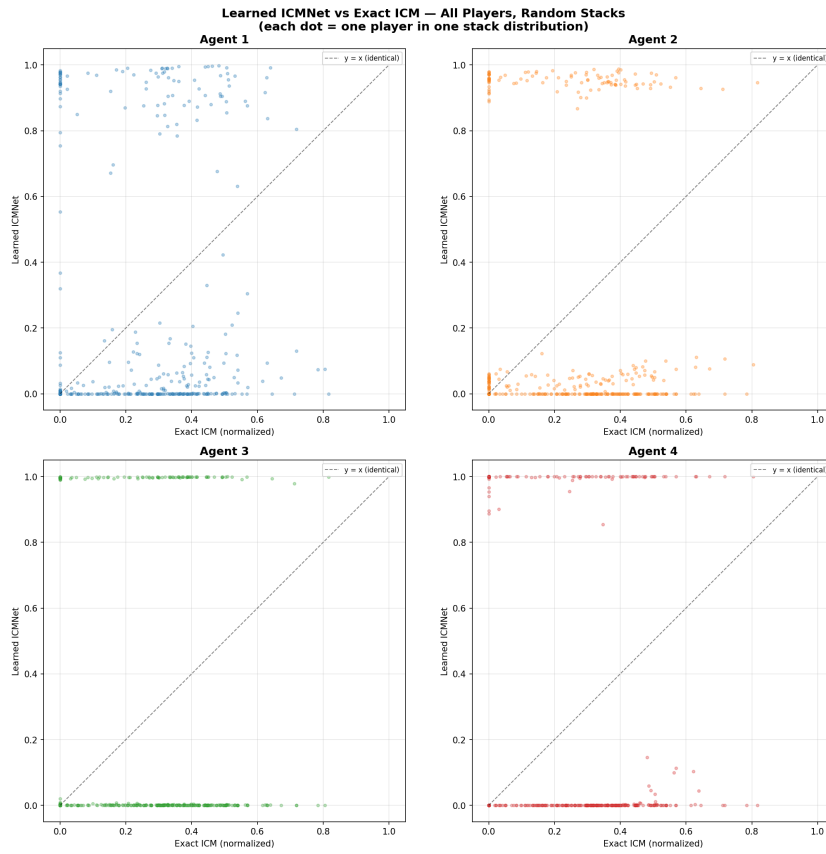


Figure 7: Learned vs exact ICM across 100 random stack distributions.

Points cluster at $y \approx 0$ and $y \approx 1$ (horizontal bands) rather than along the diagonal. No correlation between learned and exact ICM. The pattern is identical to the 6.3M analysis.

4.3 Decision Impact

The DQN is fed learned ICM (left column) vs exact ICM (center) and the push/fold decisions are compared (right = difference). Greedy action, position-averaged, 25BB.

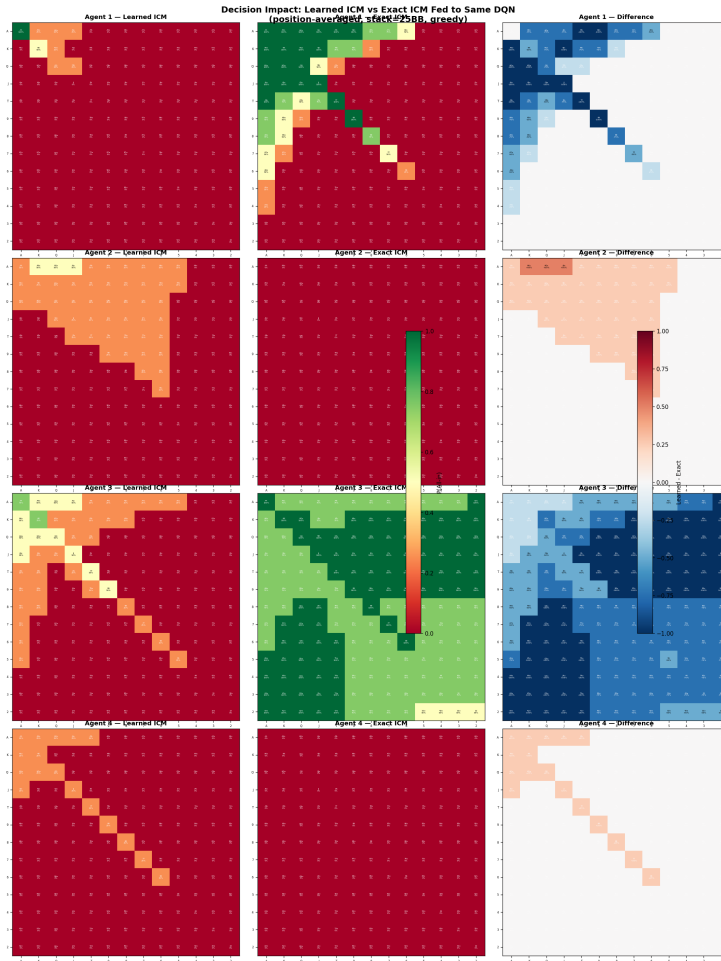


Figure 8: Decision impact — learned ICM vs exact ICM at 9.3M.

Agent	Push agreement	Mean diff	Observation
Agent 1	141/169 (83.4%)	0.175	Moderate divergence; exact ICM shifts some hands
Agent 2	169/169 (100.0%)	0.077	Identical decisions despite different ICM values
Agent 3	6/169 (3.6%)	0.780	MAJOR divergence; learned ICM pushes wide, exact folds
Agent 4	169/169 (100.0%)	0.025	Identical decisions; DQN ignores ICM entirely

Agent 3 is a critical finding. At 9.3M (vs 98.8% agreement at 6.3M), Agent 3 now shows only 3.6% agreement. This means its DQN has become **dependent** on the collapsed ICM output — it has learned to interpret the constant $[0, 0, 1, 0]$ vector as a signal to push wider. When exact ICM is substituted (a different 4-dim vector), the DQN's decisions change dramatically. The Agent 3 range with exact ICM is extremely tight (center column), while with learned ICM it pushes at 8%. The collapsed ICM acts as an accidental "push more" bias that the DQN has adapted to.

Agents 2 and 4 show the opposite: 100% agreement, meaning their DQNs have fully absorbed the constant ICM bias into their weights and are invariant to the ICM input.

4.4 Property Comparison

Monotonicity (bigger stack $\rightarrow$ higher equity):

	Exact ICM	Agent 1	Agent 2	Agent 3	Agent 4
Monotonicity	100.0%	52.5%	46.2%	45.3%	51.0%

All agents ~50% — equivalent to random. No improvement from 6.3M.

Rank ordering agreement and Spearman rho:

Metric	Agent 1	Agent 2	Agent 3	Agent 4
Rank match	3.0%	6.0%	1.0%	8.0%
Spearman rho	0.082	-0.100	-0.078	0.004

Near chance level (4.2%) for rank match. No correlation.

Key scenario outputs at 9.3M:

Each agent's ICMNet outputs a fixed one-hot vector regardless of stacks:

Agent	Collapsed to	Equal stacks output	One dominant output
Agent 1	Player 3	[0.000 0.022 0.000 0.978]	[0.612 0.001 0.000 0.386]
Agent 2	Player 2	[0.035 0.000 0.965 0.000]	[0.107 0.000 0.893 0.000]
Agent 3	Player 2	[0.000 0.000 0.998 0.001]	[0.000 0.002 0.996 0.002]
Agent 4	Player 0	[1.000 0.000 0.000 0.000]	[1.000 0.000 0.000 0.000]

Agent 1 shows slight sensitivity to extreme stack imbalances (the "one dominant" scenario shifts mass from player 3 to player 0), but this is insufficient for meaningful ICM estimation.

5. Key Findings and Interpretation

5.1 Strategy is still evolving at 9.3M

The V-shaped push frequency curves of Agents 2 and 3 (tighten then loosen) indicate the self-play dynamics have not reached a Nash equilibrium. The agents are still adapting to each other's strategies. Agent 4 acts as the exploiter, remaining loose against tight opponents. The tightening of Agent 4 from 46% to 15% suggests the other agents have partially adapted, making exploitation harder.

5.2 Positional awareness is emerging

The progression plot shows position lines spreading apart after ~6M for Agents 2 and 4. Later position (acting last, more information) correlates with higher push frequency — this is correct poker strategy. Agents 1 and 3 show less positional differentiation.

5.3 ICM collapse is permanent without intervention

The ICMNet remained collapsed from 6.3M through 9.3M for all agents. 3 million additional training tournaments produced no recovery. This confirms the collapse is a stable equilibrium caused by softmax saturation and the indirect training signal. The collapse will not self-resolve with more training.

5.4 Agent 3 developed ICM-dependent behavior

Between 6.3M (98.8% decision agreement) and 9.3M (3.6% agreement), Agent 3's DQN learned to use its collapsed ICM output $[0,0,1,0]$ as a meaningful feature. This is not true ICM learning — the DQN has simply adapted to treat this constant vector as a "push wider" signal. When replaced with exact ICM values, the agent becomes extremely tight. This is an important cautionary finding: the DQN can become dependent on arbitrary constant features, creating fragile behavior.

5.5 DQN base state carries the strategic intelligence

For Agents 1, 2, and 4, push/fold decisions are 83-100% identical regardless of ICM input. The 8-dim base state (hand ranks, suited flag, stack/BB, active players, shortest flag, position, call/pot) is sufficient for the agents' learned strategies. The DQN's own `stack_norm` feature implicitly captures stack-depth dependent play.

6. Recommendations

6.1 Continue training

Strategy is still evolving. The V-shaped trajectories of Agents 2-3 suggest more training could yield further strategic refinement. Consider running to 15-20M to see if push frequencies stabilize.

6.2 Fix the ICM module

The current indirect training approach has failed. Options, in order of implementation simplicity:

1. **Supervised pre-training:** Pre-train ICMNet against exact Malmuth-Harville targets, then freeze or fine-tune with small LR.
2. **Auxiliary loss:** Add $L = L_{TD} + \lambda \cdot L_{ICM}$ during RL training, where L_{ICM} is MSE against exact ICM.

3. **Remove softmax:** Replace with linear or ReLU output to prevent saturation collapse.
4. **Direct injection:** Feed exact ICM values directly as features, bypassing the network entirely.
5. **Remove ICMNet:** Use the simpler 8-dim DQNAgent if ICM awareness is not needed.

6.3 Monitor Agent 3's ICM dependency

Agent 3's newly developed dependency on the collapsed ICM vector is a fragility risk. If this agent were deployed or its weights transferred, the constant ICM bias would need to be preserved. This underscores the need to fix the ICM module before production use.

Reproducibility

```
# Range snapshots
python analysis/range_snapshots.py --milestones 2000000 4500000 7000000 9300000

# Push frequency progression
python analysis/range_progression.py --max-tournaments 9300000

# ICM comparison
python analysis/icm_compare.py --max-tournaments 9300000
```

All evaluations use checkpoints from run 20260416_155300_916856. Random stack distributions use seed=42 for reproducibility.